SelectKBest Algorithm

All the needed libraries are being imported using the below code.

```
import pandas as pd
from sklearn.model_selection import train_test_split
import time
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
import pickle
import matplotlib.pyplot as plt
```

The below piece of code selects the best features using chi2 test.

```
def selectkbest(in_x,out_y,n):
    test=SelectKBest(score_func=chi2,k=n)
    fit1=test.fit(in_x,out_y)
    selectk_features=fit1.transform(in_x)
    return selectk_features
```

Training and test data is being segregated using the below code.

```
def split_scalar(in_x,out_y):
    x_train,x_test,y_train,y_test=train_test_split(in_x,out_y,test_size=0.25,random_state=0)
    sc=StandardScaler()
    x_train=sc.fit_transform(x_train)
    x_test=sc.transform(x_test)
    return x_train,x_test,y_train,y_test
```

The confusion matrix is being calculated using the below code.

```
def cm_prediction(classifier,x_test):
    y_pred=classifier.predict(x_test)
    #confusion matrix
    from sklearn.metrics import confusion_matrix
    cm=confusion_matrix(y_test,y_pred)

    from sklearn.metrics import accuracy_score
    from sklearn.metrics import classification_report
    Accuracy=accuracy_score(y_test,y_pred)
    report=classification_report(y_test,y_pred)
    return classifier,Accuracy,report,x_test,y_test,cm
```

The below different function are used to create different machine learning models.

```
def logistic(x_train,y_train,x_test):
    from sklearn.linear_model import LogisticRegression
    classifier=LogisticRegression(random_state=0)
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

```
def svm_linear(x_train,y_train,x_test):
    from sklearn.svm import SVC
    classifier=SVC(kernel='linear',random_state=0)
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

```
def svm_NL(x_train,y_train,x_test):
    from sklearn.svm import SVC
    classifier=SVC(kernel='rbf',random_state=0)
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

```
def Navie(x_train,y_train,x_test):
    from sklearn.naive_bayes import GaussianNB
    classifier=GaussianNB()
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

```
def decision(x_train,y_train,x_test):
    from sklearn.tree import DecisionTreeClassifier
    classifier=DecisionTreeClassifier(criterion='entropy',random_state=0)
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

```
def random(x_train,y_train,x_test):
    from sklearn.ensemble import RandomForestClassifier
    classifier=RandomForestClassifier(n_estimators=10,criterion='entropy',random_state=0)
    classifier.fit(x_train,y_train)
    classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
    return classifier, Accuracy, report, x_test,y_test,cm
```

The below piece of code creates a dataframe for displaying the chisquare for each machine learning model.

```
def selectk_classification(acclog,accsvml,accsvmnl,accknn,accnav,accdec,accrf):
    dataframe=pd.DataFrame(index=['ChiSquare'],columns=['Logistic','SVM1','SVMnl','KNN','Navie','Decision','Random'])
    for number,idex in enumerate(dataframe.index):
        dataframe['Logistic'][idex]=acclog[number]
        dataframe['SVM1'][idex]=accsvml[number]
        dataframe['SVMnl'][idex]=accsvmnl[number]
        dataframe['KNN'][idex]=accknn[number]
        dataframe['Navie'][idex]=accnav[number]
        dataframe['Decision'][idex]=accdes[number]
        dataframe['Random'][idex]=accrf[number]
    return dataframe
```

The input file is being loaded using the below code.

```
dataset1=pd.read_csv("prep.csv",index_col=None)

df2=dataset1
df2=pd.get_dummies(df2,drop_first=True)

in_x=df2.drop('classification_yes', axis=1)
out_y=df2['classification_yes']
```

The below piece of code is calling the machine learning model functions which in turn calculates the accuracy, classification report, confusion matrix and chisquare values. The dataframe is being displayed with the results.

	Logistic	SVM1	SVMnl	KNN	Navie	Decision	Random
ChiSquare	0.94	0.94	0.95	0.89	0.83	0.96	0.95

Rescursive Feature Elimination(RFE)

The below libraries are loaded using the below import statements.

```
import pandas as pd
from sklearn.model_selection import train_test_split
import time
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
import pickle
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
```

The below function creates machine learning base models(Logistic Regression, SVC, Random Forerst, Decision Tree). For each of these ML models, Feature selection is done and the result is stored in a list rfelist.

```
def rfeFeature(in_x,out_y,n):
        rfelist=[]

        log_model = LogisticRegression(solver='lbfgs')
        RF = RandomForestClassifier(n_estimators = 10, criterion = 'entropy', random_state = 0)
       # NB = GaussianNB()
        DT= DecisionTreeClassifier(criterion = 'gini', max_features='sqrt',splitter='best',random_state = 0)
        svc_model = SVC(kernel = 'linear', random_state = 0)
        #knn = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
        rfemodellist=[log_model,svc_model,RF,DT]
        for i in   rfemodellist:
            print(i)
            log_rfe = RFE(estimator=i, n_features_to_select=n)
            #Log_rfe = RFE(i, n)
            log_fit = log_rfe.fit(in_x, out_y)
            log_rfe_feature=log_fit.transform(in_x)
            rfelist.append(log_rfe_feature)
        return rfelist
```

All the other logic is same as of SelectKBest algorithm.

The training and test dataset is being segregated using the below code. And the data is preprocessed using standard scaler method.

```
def split_scalar(in_x,out_y):
        x_train, x_test, y_train, y_test = train_test_split(in_x, out_y, test_size = 0.25, random_state = 0)
        #X_train, X_test, y_train, y_test = train_test_split(indep_X,dep_Y, test_size = 0.25, random_state = 0)

        #Feature Scaling
        #from sklearn.preprocessing import StandardScaler
        sc = StandardScaler()
        X_train = sc.fit_transform(x_train)
        X_test = sc.transform(x_test)

        return x_train, x_test, y_train, y_test
```

The below method finds the confusion matrix

```
def cm_prediction(classifier,x_test):
     y_pred = classifier.predict(x_test)

        # Making the Confusion Matrix
     from sklearn.metrics import confusion_matrix
     cm = confusion_matrix(y_test, y_pred)

     from sklearn.metrics import accuracy_score
     from sklearn.metrics import classification_report

     Accuracy=accuracy_score(y_test, y_pred )

     report=classification_report(y_test, y_pred)
     return  classifier,Accuracy,report,x_test,y_test,cm
```

The different ML models are created using the below code.

```
def logistic(x_train,y_train,x_test):

        from sklearn.linear_model import LogisticRegression
        classifier = LogisticRegression(random_state = 0)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def svm_linear(x_train,y_train,x_test):

        from sklearn.svm import SVC
        classifier = SVC(kernel = 'linear', random_state = 0)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def svm_NL(x_train,y_train,x_test):

        from sklearn.svm import SVC
        classifier = SVC(kernel = 'rbf', random_state = 0)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def Navie(x_train,y_train,x_test):

        from sklearn.naive_bayes import GaussianNB
        classifier = GaussianNB()
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def knn(x_train,y_train,x_test):


        from sklearn.neighbors import KNeighborsClassifier
        classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def Decision(x_train,y_train,x_test):


        from sklearn.tree import DecisionTreeClassifier
        classifier = DecisionTreeClassifier(criterion = 'entropy', random_state = 0)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

```
def random(x_train,y_train,x_test):


        from sklearn.ensemble import RandomForestClassifier
        classifier = RandomForestClassifier(n_estimators = 10, criterion = 'entropy', random_state = 0)
        classifier.fit(x_train, y_train)
        classifier,Accuracy,report,x_test,y_test,cm=cm_prediction(classifier,x_test)
        return  classifier,Accuracy,report,x_test,y_test,cm
```

The below method creates a dataframe and the data for the dataframe is being populated using the for loop.

```
def rfe_classification(acclog,accsvml,accsvmnl,accknn,accnav,accdes,accrf):

    rfedataframe=pd.DataFrame(index=['Logistic','SVC','Random','DecisionTree'],columns=['Logistic','SVMl','SVMNl',
                                                                                        'KNN','Navie','Decision','Random'])

    for number,idex in enumerate(rfedataframe.index):

        rfedataframe['Logistic'][idex]=acclog[number]
        rfedataframe['SVMl'][idex]=accsvml[number]
        rfedataframe['SVMNl'][idex]=accsvmnl[number]
        rfedataframe['KNN'][idex]=accknn[number]
        rfedataframe['Navie'][idex]=accnav[number]
        rfedataframe['Decision'][idex]=accdes[number]
        rfedataframe['Random'][idex]=accrf[number]
    return rfedataframe
```

```
dataset1=pd.read_csv("prep.csv",index_col=None)
df2=dataset1
df2 = pd.get_dummies(df2, drop_first=True)

in_x=df2.drop('classification_yes', axis=1)
out_y=df2['classification_yes']


rfelist=rfeFeature(in_x,out_y,3)
rfelist
```

For each of the base models(Logistic Regression, SVC, Decision Tree and Random Forest), the different ML models(Logistic Regression, SVM linear, SVM non linear, KNN, Navie Bayes, Decision Tree and Random Forest) are created and the accuracy for each combination is calculated.

	Logistic	SVMl	SVMNl	KNN	Navie	Decision	Random
Logistic	0.94	0.94	0.94	0.94	0.94	0.94	0.94
SVC	0.87	0.87	0.87	0.64	0.87	0.87	0.87
Random	0.94	0.95	0.94	0.93	0.86	0.91	0.94
DecisionTree	0.95	0.95	0.9	0.93	0.77	0.94	0.97